Séance 5 · Tester, sécuriser et appeler l'API

MaintAI reste l'exemple. En séance 4, le modèle est devenu une API. Aujourd'hui, vous essayez de la casser avec des tests, vous la protégez par une clé, puis une page web l'appelle (TP2). Le TP2 se range dans `TP/TP2`, à côté de `TP/TP0` et `TP/TP1`.

Étape	Ce que vous faites	Durée
1. TP1	finir et rendre le TP1 (pull request, tag `tp1`)	25 min
2. Tester	casser son API : cinq requêtes, cinq tests	40 min
3. Sécuriser	ruff, la clé API, `.env`, CORS	35 min
4. TP2, sprint 1	la clé API protège `/v1`, jusqu'à 18 tests verts	45 min
5. TP2, sprint 2	une page web appelle l'API ; un client Java	45 min
6. Rendre	pull request, tag `tp2`, ticket de sortie	20 min

À rendre : le TP1 fusionné et tagué `tp1` (avec son `COMPTE_RENDU.md`) en début de séance ; le TP2 dans `TP/TP2`, avec son `COMPTE_RENDU.md` rempli, par une pull request et le tag `tp2`.

1 · Finir et rendre le TP1

En groupe, 15 minutes. Dans `TP\TP1`, `python -m pytest` doit donner **13 passed**, et `COMPTE_RENDU.md` doit être rempli. Puis :

```
git push                        # la branche tp1
# GitHub : Compare & pull request ; un coéquipier relit, commente, approuve
# Merge pull request (Create a merge commit)
git switch main
git pull
git tag tp1
git push --tags
```

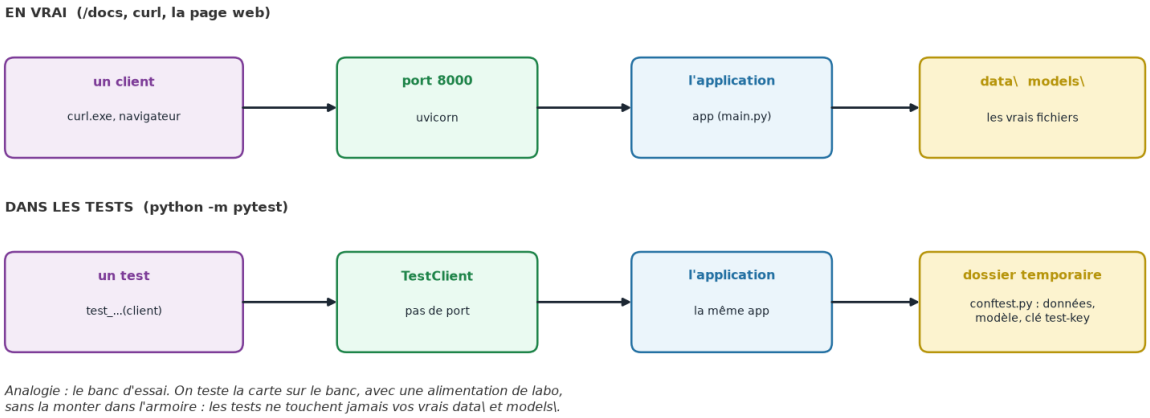
Le relecteur vérifie : `pytest` passe chez lui ; ni données, ni modèle, ni `.venv` dans les changements ; l'ADR 002 et `COMPTE_RENDU.md` sont remplis ; au moins un commentaire.

À montrer : la pull request fusionnée et le tag `tp1` sur GitHub.

2 · Tester une API

Un test est une **question précise** avec la **réponse attendue** : « si j'envoie 500 °C, l'API répond-elle 422 ? ». Jusqu'ici les tests étaient fournis ; aujourd'hui vous en écrivez.

TestClient : tester sans serveur. Dans les tests, on ne lance pas uvicorn. `TestClient` envoie les requêtes directement à l'application, sans port. Les *fixtures* de `tests/conftest.py` préparent un dossier temporaire avec des données, un modèle et une clé de test : les tests ne touchent jamais vos vrais `data\` et `models\`.



Analogie : le banc d'essai. On teste la carte sur le banc, avec une alimentation de labo, sans la monter dans l'armoire : les tests ne touchent jamais vos vrais data\ et models\.

```
VALID = {"temperature_c": 60.0, "vibration_mm_s": 2.0, "load_pct": 50.0, "runtime_hours": 5000}
def test_500_degres_est_refuse(client):         # client : une fixture de conftest.py
    r = client.post("/v1/predict", json={**VALID, "temperature_c": 500})
    assert r.status_code == 422                 # la réponse attendue
```

Morceau	Sens
`def test_...(client)`	pytest lance toute fonction dont le nom commence par `test_` ; il lui donne la fixture `client`
`{**VALID, "temperature_c": 500}`	une copie de `VALID` où une seule valeur change
`client.post(url, json=...)`	la requête, comme `/docs` ou `curl.exe` l'enverraient
`assert ... == 422`	si c'est faux, le test échoue et pytest affiche les deux valeurs

Ce qu'on teste	Comment	Exemple
les données	pytest et `make_dataset()`	les colonnes attendues
le modèle	entraînement temporaire	il bat la baseline
le cas nominal	TestClient	un relevé normal donne 200
les entrées refusées	TestClient	500 °C donne 422
la sécurité	TestClient et un en-tête	sans clé, 401 (TP2)
le navigateur	une requête OPTIONS	une origine inconnue est refusée (TP2)

Activité 1 · Casser son API

En groupe, 25 minutes, dans `TP\TP1`.

1. Écrivez cinq requêtes qui devraient être refusées, et **prévoyez** le code de chacune avant de l'envoyer :

Cas	Requête (corps ou URL)	Prévu	Obtenu
champ manquant			
valeur absurde			
texte au lieu d'un nombre			
champ en trop			
mauvaise route			

2. Lancez `uvicorn maintai.api.main:app --reload`, envoyez-les depuis `/docs`, remplissez « Obtenu ».
3. Transformez-les en tests dans `TP\TP1\tests\test_casser.py`. Un seul test, cinq cas, grâce à `parametrize` :

pytest lance la fonction une fois par ligne de la liste, et nomme chaque cas dans le rapport.

```
import pytest
HEADERS = {"X-API-Key": "test-key"}   # la clé des tests (conftest.py) ; inutile en TP1, nécessaire en TP2
VALID = {"temperature_c": 60.0, "vibration_mm_s": 2.0, "load_pct": 50.0, "runtime_hours": 5000}
CAS_REFUSES = [
    # (description, url, corps, code attendu)
    ("valeur absurde", "/v1/predict", {**VALID, "load_pct": 250}, 422),
    # ... vos quatre autres cas
]
@pytest.mark.parametrize("description, url, corps, attendu", CAS_REFUSES)
def test_requete_refusee(client, description, url, corps, attendu):
    assert client.post(url, json=corps, headers=HEADERS).status_code == attendu
```

Pour un champ manquant : `SANS_CHARGE = {k: v for k, v in VALID.items() if k != "load_pct"}` (`VALID` sans une clé). `python -m pytest -v` affiche le nom de chaque cas.

4. Une requête acceptée alors qu'elle devrait être refusée ? Corrigez les bornes dans `schemas.py` et gardez le test.

À montrer : `python -m pytest` avec vos cinq tests : **18 passed** (13 + 5).

3 · La qualité du code : ruff

`pytest` *exécute* le code et vérifie ce qu'il fait. `ruff` *lit* le code sans l'exécuter et signale ce qui est mal écrit : un import inutile (F401), des imports mal rangés (I001), une variable jamais utilisée. Dans `TP\TP1`, environnement activé :

```
pip install ruff              # une seule fois
ruff check .                  # liste les problèmes ; [*] = corrigeable seul
ruff check . --fix            # corrige ce qui est sans risque
ruff format .                 # met le code en forme
python -m pytest              # vérifie que rien n'est cassé
```

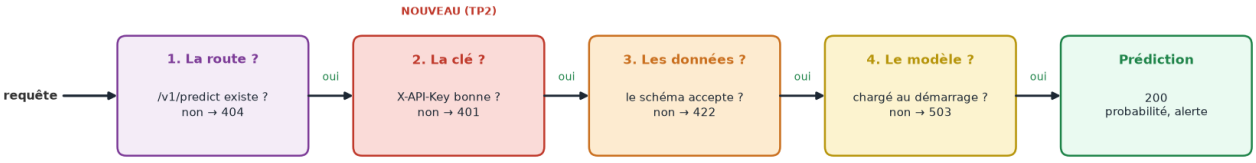
Le code d'un TP1 bien écrit donne `All checks passed!`. Règle : `ruff check .` avant chaque commit, `python -m pytest` avant chaque push. En séance 8, GitHub le fera à chaque push (l'intégration continue).

Attention dans `TP\TP2` : le starter importe déjà les outils dont vous aurez besoin (`Depends`, `secrets`...). Tant que le code n'est pas écrit, ruff les signale « imported but unused » (F401), et `ruff check . --fix` **les supprimerait**. Ne lancez jamais `--fix` dans `TP\TP2` avant d'avoir fini le sprint 1.

4 · Sécuriser une API

Question	Réponse dans MaintAI
Qui appelle ?	*authentification* : une clé API dans l'en-tête `X-API-Key` ; sinon **401**
A-t-il le droit ?	*autorisation* : sinon **403** (hors TP2)
Où sont les secrets ?	dans `.env`, lu au démarrage ; jamais dans le code ni dans Git
Depuis quel site ?	CORS : la liste des origines autorisées, dans `.env`

L'ordre des contrôles. Une requête passe une suite de contrôles ; le premier qui échoue donne la réponse.



Des protections en série, comme sectionneur, disjoncteur, relais : la première qui déclenche coupe le circuit. Sans clé, l'API ne regarde même pas les données : 500 °C sans clé donne 401, pas 422.

La clé voyage dans un en-tête, jamais dans l'URL : une URL est enregistrée partout (journaux du serveur, historique du navigateur). Avec `curl.exe`, la clé est un en-tête de plus :

```
curl.exe -i -X POST http://localhost:8000/v1/predict -H "Content-Type: application/json" -H "X-API-Key: ma-cle" -d "@releve.json"
```

`-i` affiche aussi la ligne de statut. Sans clé ou avec une fausse clé : `HTTP/1.1 401 Unauthorized` et `{"detail":"Clé API absente ou invalide"}`.

Où mettre la clé ?

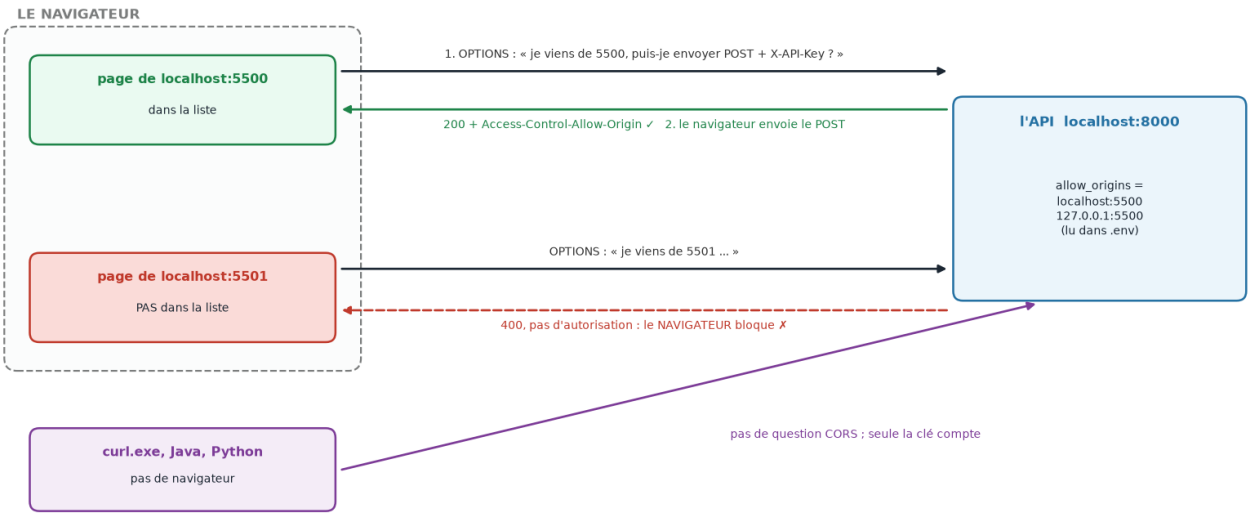


Analogie : .env.example est le schéma de câblage (quelles bornes brancher) ; .env est la clé de l'armoire électrique : chacun garde la sienne, on ne la photocopie pas.

Endroit	Bonne idée ?	Pourquoi
dans le code	non	elle finit sur GitHub, pour toujours
`.env`	oui	lu au démarrage (`--env-file .env`), ignoré par Git
`.env.example`	oui	commité, avec une fausse valeur : il dit quoi configurer
une variable du serveur	oui	en production (séance 8)

Une clé commitée une seule fois reste dans l'historique de Git, même si on supprime le fichier ensuite. Il faut alors la changer. Avant chaque commit : `git status`, et `.env` ne doit pas apparaître.

CORS : le navigateur demande la permission. Une *origine*, c'est un schéma, un domaine et un port : `http://localhost:5500` et `http://localhost:8000` sont deux origines différentes. Quand une page appelle une autre origine, le navigateur envoie d'abord une requête `OPTIONS` (« puis-je ? »). Si l'API ne répond pas `Access-Control-Allow-Origin`, le navigateur bloque.

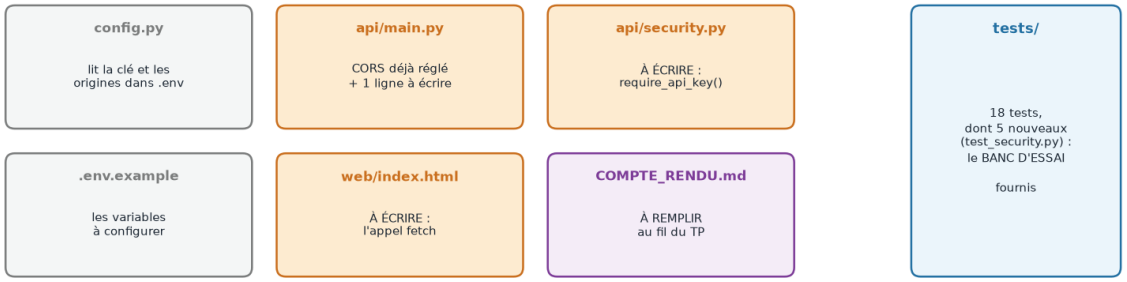


CORS est une règle DU NAVIGATEUR : il protège l'utilisateur contre une page qui utiliserait son navigateur pour appeler l'API. Ce n'est pas une protection de l'API : curl passe. La clé reste indispensable.

TP2 · Clé API, CORS et client web

Séance 5, fin en début de séance 6 · thèmes 2.3, 2.4 et 5.1 du syllabus

Contexte. L'API MaintAI ne doit répondre qu'aux programmes autorisés : chacun envoie une clé dans l'en-tête `X-API-Key`. La clé ne doit jamais apparaître dans le code ni dans Git. Une page web servie sur `http://localhost:5500` doit pouvoir appeler l'API ; une page servie ailleurs, non.



Gris : fourni, à lire seulement. Orange : à écrire. Violet : à remplir. Bleu : le banc d'essai.

Mise en place dans le dépôt : TP/TP2 (10 min)

Le starter du TP2 contient le TP1 corrigé et les nouveaux fichiers. Il se range dans `TP\TP2` : votre propre TP1 reste intact dans `TP\TP1`.

dépôt du groupe	sur GitHub, cloné chez chaque membre
TP\TP0\	README.md, un rapport Prenom_Nom.md par membre (S4)
TP\TP1\	l'API du TP1 (S4)
tests\test_casser.py	NOUVEAU : activité 1, cinq requêtes refusées
.venv\	l'environnement Python (pas dans Git), réutilisé par le TP2
TP\TP2\	le contenu de S5_maintai-starter-tp2.zip
maintai\api\security.py	À ÉCRIRE : la clé API
web\index.html	À ÉCRIRE : la page qui appelle l'API
.env.example	commité : les variables à configurer
.env	votre clé (pas dans Git)
COMPTE_RENDU.md	à remplir, rendu avec la pull request

Un dossier par TP. Le starter du TP2 contient le TP1 corrigé : votre propre TP1 reste intact dans TP\TP1.

```
# -- un membre --
git switch main
git pull                        # le TP1 fusionné
git switch -c tp2               # une branche pour le TP2
# copier le contenu de S5_maintai-starter-tp2.zip dans TP\TP2
git add TP
git commit -m "TP2 : starter"
git push -u origin tp2

# -- chaque membre --
git pull
git switch tp2
cd TP\TP2
..\TP1\.venv\Scripts\activate.bat  # l'environnement du TP1 (mêmes paquets)
python -m maintai.dataset
python -m maintai.train
python -m pytest                # attendu : 2 failed, 16 passed
```

PowerShell : `..\TP1\.venv\Scripts\Activate.ps1`. Pas d'environnement dans `TP\TP1` ? `python -m venv .venv`, activez-le, puis `pip install -r requirements.txt` dans `TP\TP2`. Les 2 échecs : sans clé et avec une fausse clé, l'API répond encore 200.

Sprint 1 · La clé API (35 min)

1. **`maintai/api/security.py`** : le corps de `require_api_key()`. Elle compare la clé reçue à `get_settings().api_key` avec `secrets.compare_digest` ; `HTTPException` 401 si la clé est absente ou fausse ; 500 si le serveur n'a pas de clé configurée. `pytest` donne **encore 2 failed** : la fonction existe, mais rien ne l'appelle.
2. **`maintai/api/main.py`** : une seule ligne change. Ajoutez au router l'argument `dependencies=[Depends(require_api_key)]` : chaque route `/v1` appelle `require_api_key` avant son propre code. `pytest` : **18 passed**.
3. La clé de votre serveur : `copy .env.example .env`, ouvrez `.env`, remplacez `changez-moi` par une clé de votre choix, puis :

   ```
   uvicorn maintai.api.main:app --reload --env-file .env
   ```

4. `/docs` : un champ `x-api-key` apparaît. Sans clé : 401 ; avec la clé de `.env` : 200 ; `/health` reste public.

À montrer : 18 passed, et le 401 sans clé dans `/docs`.

À écrire	Pistes, sans la solution
lire la clé du client	elle arrive en paramètre : `x_api_key: str | None = Header(default=None)` lit l'en-tête `X-API-Key` ; `None` s'il est absent
la clé attendue	`get_settings().api_key` ; elle vaut `None` si `.env` n'a pas été chargé
comparer	`secrets.compare_digest(a, b)` plutôt que `a == b` : `==` s'arrête au premier caractère faux, et la durée de la réponse trahirait combien de caractères étaient justes
refuser	`raise HTTPException(status_code=401, detail="...")`
protéger le router	`Depends(require_api_key)` : on donne la fonction, **sans parenthèses** ; FastAPI l'appelle à chaque requête

Sprint 2 · Le client web (35 min)

1. **`web/index.html`** : dans le gestionnaire du bouton, envoyez `reading` en POST JSON à `<url>/v1/predict` avec les en-têtes `Content-Type` et `X-API-Key`. La forme d'un appel `fetch` :

```
try {
    const resp = await fetch(adresse, {              // adresse : <url>/v1/predict
        method: "POST",
        headers: { "Content-Type": "application/json", "X-API-Key": cle },
        body: JSON.stringify(reading),              // l'objet devient du texte JSON
    });
    const data = await resp.json();                 // la réponse, en objet
    if (!resp.ok) { ... }                          // 401, 422, 503 : afficher resp.status
    // sinon : data.failure_probability et data.alert
} catch (e) { ... }                    // API arrêtée, ou CORS qui bloque
```

Affichez le résultat avec `show("ok", ...)`, `show("alert", ...)` ou `show("err", ...)`.

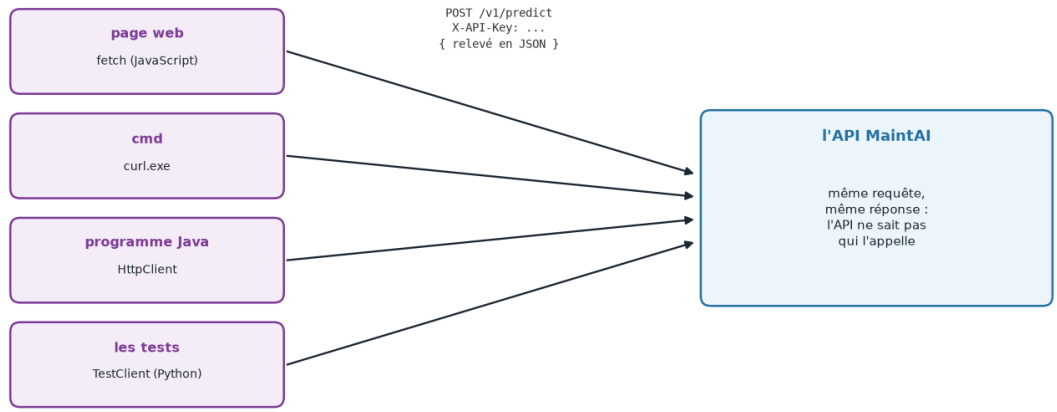
2. Second terminal, dans `TP\TP2`, même environnement : `python -m http.server 5500 -d web`, puis ouvrez `http://localhost:5500`. Pas de double-clic sur le fichier : son origine serait `null`, refusée.
3. Saisissez la clé : un relevé normal, puis 500 °C, puis une fausse clé. Notez ce qui s'affiche :

---

4. **Expérience CORS.** Arrêtez le serveur de la page (Ctrl+C), relancez-le sur le port 5501, ouvrez `http://localhost:5501`. Que disent la page et la console (F12) ? Pourquoi `curl.exe` marche-t-il quand même ?

À montrer : la page qui affiche une probabilité, et l'erreur CORS du port 5501.

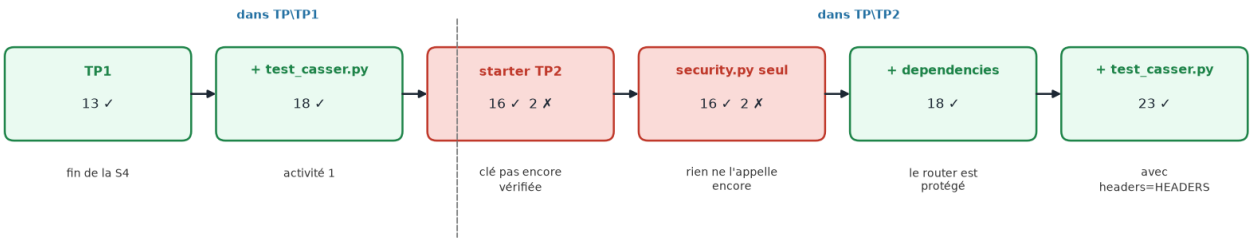
D'autres clients. Une page web, `curl.exe`, un programme Java, les tests : tous envoient la même requête. L'API ne sait pas qui l'appelle ; seul le contrat compte.



Comme un connecteur normalisé : n'importe quel appareil qui respecte le brochage (le contrat) peut s'y brancher.

Les tests de l'activité 1, maintenant que l'API exige une clé

Copiez `TP\TP1\tests\test_casser.py` dans `TP\TP2\tests`. La clé est vérifiée avant les données : sans `headers=HEADERS`, vos cas 422 reçoivent 401 et échouent. Avec la clé : **23 passed**.



Chaque chiffre a été vérifié en exécutant le code. Un test rouge dit précisément ce qui manque.

Décision : l'ADR 003

Dans `docs/adr/003-securite.md` : **où transmettre la clé** (en-tête, URL ou corps ?) et **quelles origines autoriser** (une liste ou `*` ? configurée où ?). Pour chaque décision, ce que l'autre option ferait perdre.

Le compte rendu : COMPTE_RENDU.md

Le starter contient `COMPTE_RENDU.md`, dans `TP\TP2`. Remplissez-le au fil de la séance : vos cinq requêtes de l'activité 1, les sorties de `pytest` et de `ruff`, votre `require_api_key()`, les codes obtenus pour six requêtes, votre `fetch`, l'expérience CORS, cinq questions de compréhension et le lien de la pull request.

Rendre le TP2 (15 min)

```
git status                    # .env ne doit PAS apparaître
git push                      # la branche tp2
# GitHub : Compare & pull request ; un coéquipier relit, commente, approuve
# Merge pull request (Create a merge commit)
git switch main
git pull
git tag tp2
git push --tags
```

Le relecteur vérifie : `pytest` passe chez lui ; **aucun fichier `.env`** dans les changements ; l'ADR 003 et `COMPTE_RENDU.md` sont remplis ; au moins un commentaire.

Critères de réussite. 18 tests passent (23 avec `test_casser.py`) ; sans clé, 401 ; `/health` reste public ; la page prédit depuis le port 5500 et échoue depuis 5501 ; `.env` n'est pas dans Git ; la pull request est fusionnée et le tag `tp2` existe.

En cas de problème

Symptôme	Solution
`ruff` introuvable	`pip install ruff` dans l'environnement activé
`NameError: name 'Depends'` (ou `secrets`)	`ruff --fix` a supprimé l'import : remettez la ligne d'import du starter
`..\TP1\.venv` introuvable	créez un environnement dans `TP\TP2` : `python -m venv .venv`, puis `pip install -r requirements.txt`
toutes les requêtes : 500 « n'est pas configurée »	relancez uvicorn avec `--env-file .env`
bonne clé, pourtant 401	la clé saisie diffère de `.env` ; ou uvicorn n'a pas été relancé après la modification de `.env`
`Depends(require_api_key())` : erreur au lancement	passez la fonction, sans parenthèses
`.env` apparaît dans `git status`	il doit être ignoré : vérifiez le `.gitignore` de `TP\TP2` ; ne le commitez jamais
`.env` déjà poussé	`git rm --cached .env`, commit, push, et **changez la clé**
vos tests de l'activité 1 échouent en TP2 (401)	ajoutez `headers=HEADERS` à `client.post`
la page affiche « API injoignable »	uvicorn est arrêté, ou la page n'est pas servie sur le port 5500 (CORS)
la page ouverte par double-clic ne marche pas	servez-la : `python -m http.server 5500 -d web`
nouvelle origine dans `.env`, toujours refusée	relancez uvicorn : la liste est lue au démarrage
`curl` : `422 JSON decode error`	JSON dans un fichier : `curl.exe ... -d "@releve.json"`

× découpez ici

Ticket de sortie · Séance 5

Nom : _____ Groupe : _____

1. Quelle différence entre 401 et 422 ?

2. Pourquoi la clé n'est-elle ni dans le code ni dans Git ?

3. Pourquoi curl fonctionne-t-il alors que la page du port 5501 est bloquée ?

4. Qu'est-ce qui reste flou pour vous ?